

Residue-Class Tiling as a Finite Analogue for Sparse Feature Recovery

Dan Hawkley

Abstract

We study feature recovery on a controlled finite system defined by Mod30 residue classes. Using a continuous embedding of integers constrained to residues coprime with 30, we compare non-negative matrix factorization (NMF) and top- k sparse autoencoders (SAEs). We introduce metrics for structural recovery, redundancy, and capacity utilization. While NMF achieves higher coverage and efficient use of components, SAEs produce sparse and interpretable features but leave substantial latent capacity unused or redundant. These results show that sparsity-induced interpretability does not guarantee efficient representation, even with known ground truth.

1 Introduction

This work responds to recent analysis of sparse feature recovery and manifold tiling in [1], using Mod30 residue classes as a controlled finite analogue. Understanding how models recover structure is difficult when ground truth is unknown. We construct a finite system where structure is explicit.

This work is fully reproducible via an open notebook-based implementation [2].

- integers mapped under modulo 30,
- filtered to residues coprime with 30,
- yielding 8 persistent residue classes.

This provides a controlled testbed where

$$\text{true features} = \text{residue lanes.}$$

2 Mod30 Residue Structure

Let

$$R = \{1, 7, 11, 13, 17, 19, 23, 29\}.$$

These define the persistent Mod30 lanes. We embed filtered integers into a continuous space and evaluate whether factorization and sparse-learning methods recover these lanes.

3 Continuous Embedding

Each integer is mapped to:

- circular coordinates,
- lane channels with local overlap,
- additive noise.

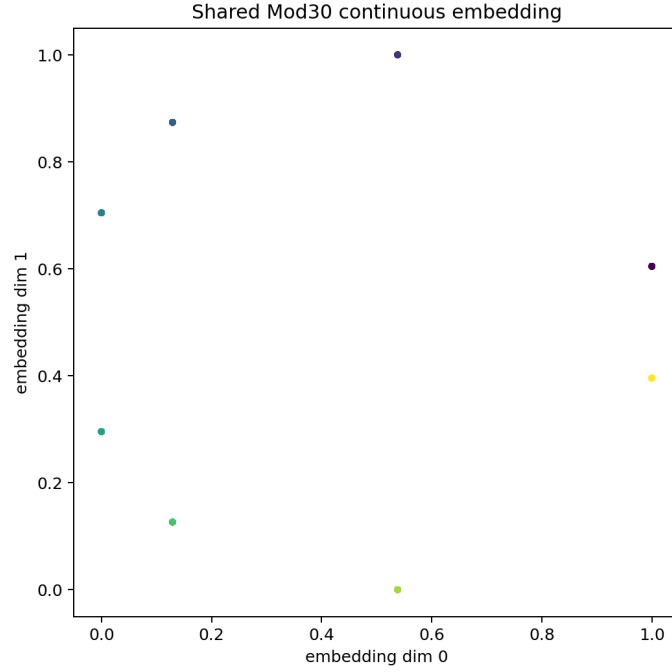


Figure 1: Continuous embedding of Mod30 residue classes.

4 Models

4.1 Non-negative Matrix Factorization

Non-negative matrix factorization (NMF) provides parts-based representations [3].

$$X \approx WH,$$

where W contains component activations and H contains learned components.

4.2 Sparse Autoencoder

Sparse representations are commonly studied in overcomplete coding frameworks [4].

$$X \longrightarrow Z \longrightarrow \text{top-}k(Z) \longrightarrow \hat{X}.$$

Only k latent units remain active per sample after the top- k mask.

5 Metrics

We evaluate both model families using the same recovery metrics:

- **Coverage**: recovered lanes divided by total lanes.
- **Purity**: alignment of learned features with residue classes.
- **Reconstruction error**: relative error between X and $\hat{X}$.
- **Redundancy**: total activations divided by captured lanes.
- **Effective features**: captured lanes divided by total components.
- **Feature status**: dead, useful, and redundant feature fractions.

6 Results

6.1 Coverage vs. Reconstruction

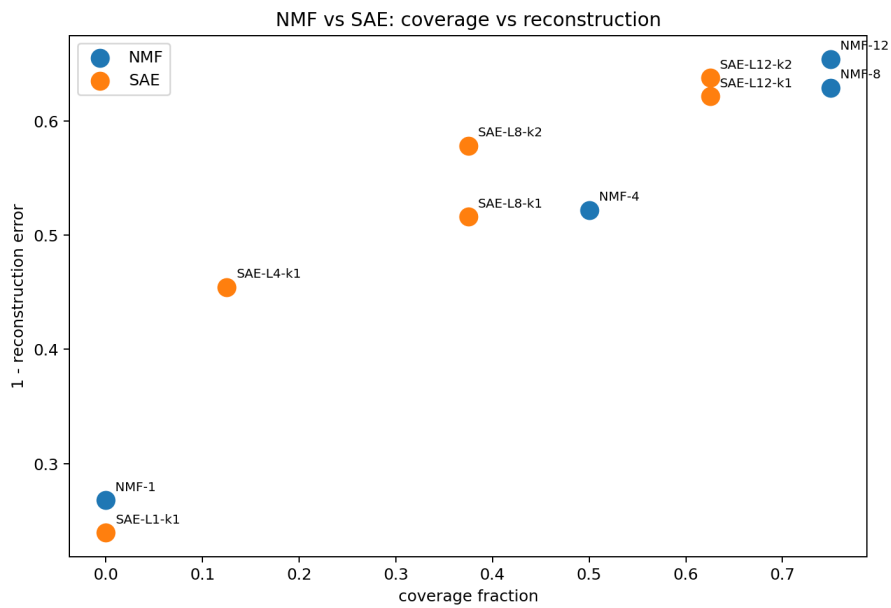


Figure 2: NMF vs. SAE: coverage and reconstruction tradeoff.

NMF achieves stronger reconstruction and higher coverage in this controlled benchmark.

6.2 Sparsity vs. Coverage

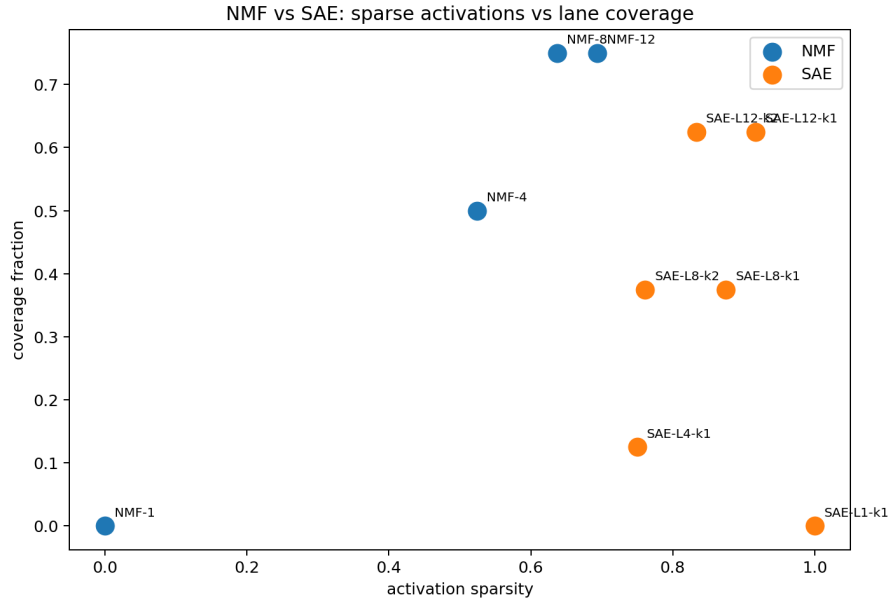


Figure 3: Sparse activations vs. structural recovery.

The SAE enforces sparse activations, but sparsity alone does not guarantee complete recovery.

6.3 Capacity Utilization

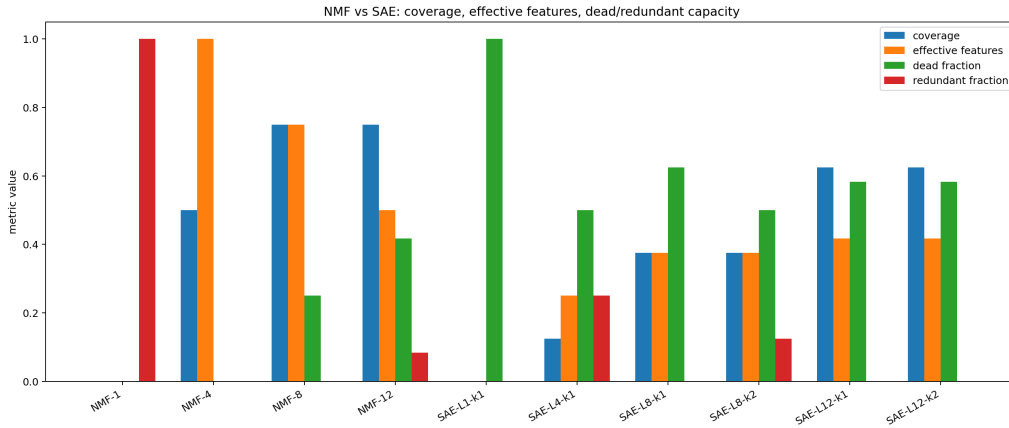


Figure 4: Effective features and dead or redundant capacity.

SAEs leave significant latent units unused or redundantly allocated.

6.4 Redundancy Tradeoff

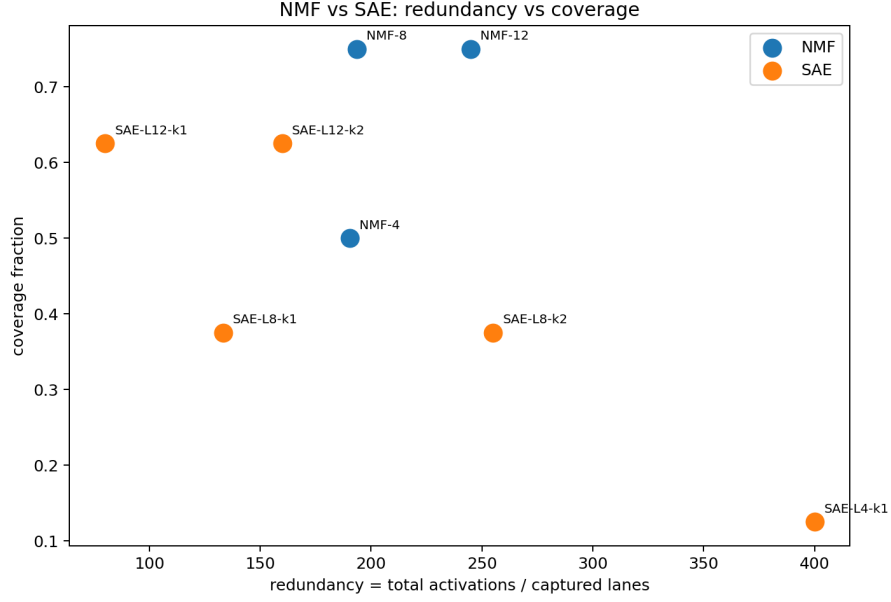


Figure 5: Redundancy vs. coverage.

6.5 Feature Alignment

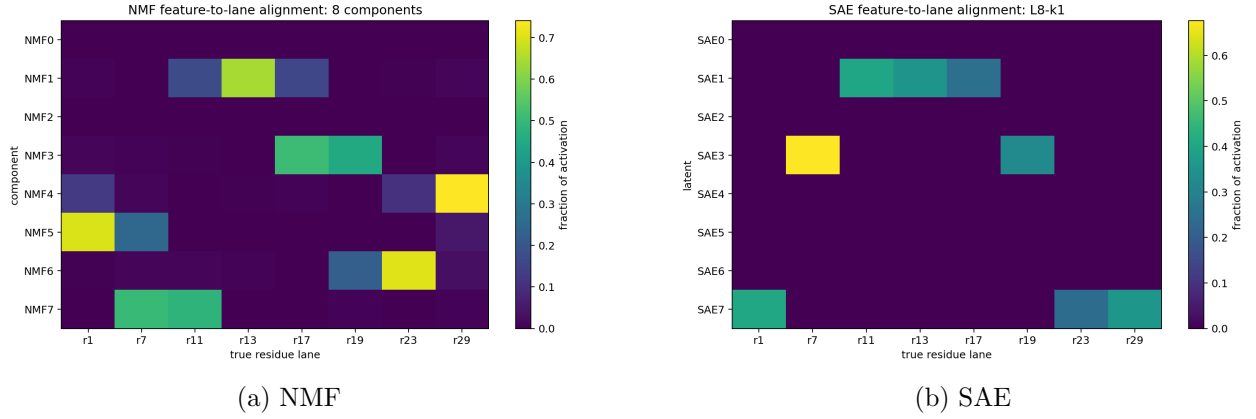


Figure 6: Feature-to-lane alignment comparison.

7 Main Result

sparsity $\implies$ interpretability but not necessarily efficient capacity.

Dense factorization efficiently uses its components, while sparse autoencoders leave significant latent capacity unused or redundant.

8 Reproducibility

All experiments, figures, and metrics are generated via publicly available notebooks:

```
01_mod30_residue_manifold
02_local_tiling_vs_global_capture
03_sparse_feature_analogy
04_mod30_sae_toy_model
05_true_sparse_autoencoder
06_nmf_vs_sae_recovery_comparison
```

The full implementation is available at:

https://github.com/thinkthoughts/mod30_manifold_tiling

See also [2].

References

- [1] Thomas Fel, Can Rager, Sheridan Feucht, Tal Haklay, Daniel Wurgaft, Siddharth Boppana, Matthew Kowal, Vasudev Shyam, Jack Merullo, Atticus Geiger, and Ekdeep Singh Lubana. Do sparse autoencoders capture manifolds?, 2026.
- [2] Dan Hawkley. Mod30 manifold tiling: Residue-class feature recovery experiments. https://github.com/thinkthoughts/mod30_manifold_tiling, 2026. Accessed: 2026.
- [3] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401:788–791, 1999.
- [4] Bruno A. Olshausen and David J. Field. Sparse coding with an overcomplete basis set. *Vision Research*, 37:3311–3325, 1997.